

Projet Algorithmique et Structures de Données

Implémentation et comparaison de structures de données pour un catalogue de bibliothèque universitaire

Licence 2 Mathématiques-Informatique
Université Iba Der Thiam de Thiès
Hawa Sonko
Aida Diop
2025-2026

Table des matières

1	Introduction	4
2	Présentation du domaine applicatif	4
3	Modélisation des données	4
4	Description des structures implémentées	5
4.1	Tableau statique	5
4.2	Tableau dynamique	5
4.3	Liste chaînée	5
5	Opérations supportées	5
6	Analyse théorique de complexité	6
6.1	Tableau statique	6
6.1.1	Insertion	6
6.1.2	Recherche par identifiant	6
6.1.3	Recherche par préfixe	6
6.1.4	Suppression	7
6.1.5	Tri par insertion	7

6.1.6	Tri rapide	7
6.1.7	Statistiques	7
6.2	Tableau dynamique	8
6.2.1	Insertion	8
6.2.2	Recherche par identifiant	8
6.2.3	Recherche par préfixe	8
6.2.4	Suppression	9
6.2.5	Tri par insertion	9
6.2.6	Tri rapide	9
6.2.7	Statistiques	9
6.3	Liste chaînée	10
6.3.1	Insertion	10
6.3.2	Recherche par identifiant	10
6.3.3	Recherche par préfixe	10
6.3.4	Suppression	10
6.3.5	Tri par insertion	11
6.3.6	Tri rapide	11
6.3.7	Statistiques	11
6.4	Tableau récapitulatif des complexités	12
7	Étude expérimentale	12
7.1	Conditions de mesure	12
7.2	Protocole de mesures	12
7.3	Interprétation des courbes	13
7.4	Graphes et interprétation	13
7.4.1	Graphique 1 : insertion	13
7.4.2	Graphique 2 : recherche	14
7.4.3	Graphique 3 : tri	15
7.4.4	Graphique 4 : insertion uniforme	15
7.4.5	Graphique 5 : recherche uniforme	16
7.4.6	Graphique 6 : tri uniforme	17
7.5	Visualisations complémentaires	18
7.5.1	Graphique 7 : comparaison en échelle logarithmique	18
7.5.2	Graphique 8 : empirique vs théorie	19
7.5.3	Graphique 9 : distribution fixe pour $n = 10^4$	19
7.5.4	Graphique 10 : histogramme des ralentissements relatifs	20
7.5.5	Graphique 11 : insertion en échelle logarithmique	20

7.5.6	Graphique 12 : recherche en échelle logarithmique	21
7.5.7	Graphique 13 : insertion empirique vs théorie	22
7.5.8	Graphique 14 : recherche empirique vs théorie	22
7.5.9	Graphique 15 : ralentissement relatif de l'insertion	23
8	Synthèse comparative	23
9	Persistance	23
10	Conclusion	24

1 Introduction

Ce rapport présente l'implémentation et l'analyse comparative de trois structures de données utilisées pour la gestion d'un catalogue de bibliothèque universitaire : le tableau statique, le tableau dynamique de pointeurs vers structures et la liste chaînée simple. L'objectif est de comparer les performances et les contraintes pratiques de ces structures sur les opérations fondamentales d'insertion, de recherche, de suppression, de tri et de calcul statistique.

Le projet est réalisé en langage C avec une organisation modulaire, des programmes séparés pour chaque structure et une compilation via Makefile. Il s'inscrit dans le cadre de l'algorithmique et des structures de données, en confrontant l'analyse théorique de la complexité aux mesures expérimentales obtenues sur des jeux de données de taille croissante.

2 Présentation du domaine applicatif

Le domaine retenu est le catalogue de bibliothèque universitaire. Les enregistrements manipulés représentent des ouvrages caractérisés par un identifiant unique, un titre, un auteur, un ISBN, une année de publication, un genre, le nombre total d'exemplaires ainsi que le nombre d'exemplaires disponibles. Ce choix permet d'illustrer un cas concret d'utilisation des structures de données en environnement de gestion d'information.

3 Modélisation des données

Le type principal utilisé est la structure `Ouvrage`, définie dans le projet comme suit :

```
struct Ouvrage {
    int id_ouvrage;
    char titre[TITRE_LEN];
    char auteur[AUTEUR_LEN];
    char isbn[ISBN_LEN];
    int annee_publication;
    char genre[GENRE_LEN];
    int nb_exemplaires_total;
    int nb_exemplaires_dispo;
    Emprunt *emprunts;
};
```

Les trois structures de données retenues sont :

- le tableau statique de structures ;
- le tableau dynamique de pointeurs vers structures ;
- la liste chaînée simple de structures.

4 Description des structures implémentées

Cette section présente les trois structures qui constituent le cœur du projet. Elles sont implémentées sur le même type d'enregistrement afin de permettre une comparaison directe de leur comportement dans des conditions identiques.

4.1 Tableau statique

Le tableau statique est une structure à capacité fixe. Il est alloué à la compilation dans un tableau de taille maximale définie par la constante `MAX`

STATIQUE. La structure stocke un nombre courant d'éléments et l'accès à un ouvrage se fait directement par indice.

4.2 Tableau dynamique

Le tableau dynamique est une structure à capacité variable. Il est alloué à l'exécution et peut être agrandi par réallocation lorsque la taille dépasse la capacité courante. Chaque case contient un pointeur vers un objet `Ouvrage`. Cela permet de garder une sémantique proche du tableau statique tout en supportant une croissance flexible.

4.3 Liste chaînée

La liste chaînée simple stocke chaque ouvrage dans un nœud contenant le champ valeur et un pointeur vers le nœud suivant. La liste n'est pas indexée : les opérations de parcours se font en suivant les pointeurs à partir de la tête. Cette structure contraste avec les tableaux en raison de l'allocation dynamique de chaque nœud et du coût d'accès séquentiel.

5 Opérations supportées

Les opérations implémentées sont les suivantes :

- insertion ;
- recherche par identifiant ;
- recherche par préfixe de titre ;
- suppression ;
- tri par insertion ;
- tri rapide ;
- calcul de moyenne, minimum et maximum ;
- persistance binaire.

6 Analyse théorique de complexité

Cette section présente les bornes asymptotiques attendues pour chaque structure et chaque opération. Les résultats sont donnés en meilleur cas, pire cas et cas moyen, puis justifiés à partir du principe algorithmique de chaque fonction.

6.1 Tableau statique

6.1.1 Insertion

L'insertion dans un tableau statique se fait en position libre à l'indice `taille`. Si la capacité n'est pas dépassée, la complexité est :

- meilleur cas : $\Omega(1)$;
- pire cas : $O(1)$;
- cas moyen : $\Theta(1)$.

La justification est la suivante : on effectue une écriture dans le tableau et une incrémentation de la `taille`. Ces deux opérations sont indépendantes du nombre d'éléments n . Il n'y a ni boucle ni décalage, donc le coût est constant.

6.1.2 Recherche par identifiant

La recherche parcourt les éléments de l'indice 0 à l'indice `taille-1` et compare l'identifiant de chaque ouvrage.

- meilleur cas : $\Omega(1)$;
- pire cas : $O(n)$;
- cas moyen : $\Theta(n)$.

Si l'élément recherché est au début, une seule comparaison suffit. Dans le pire cas, la clé se trouve en fin de tableau ou est absente, donc on effectue n comparaisons. En moyenne, on observe $\frac{n}{2}$ comparaisons pour une recherche linéaire sur n éléments.

6.1.3 Recherche par préfixe

La recherche par préfixe parcourt également tous les éléments, avec une comparaison de préfixe de chaîne pour chaque ouvrage.

- meilleur cas : $\Omega(1)$;
- pire cas : $O(n \cdot p)$;
- cas moyen : $\Theta(n \cdot p)$,

avec p la longueur du préfixe. En pratique, la comparaison de chaînes s'arrête dès la première différence ou jusqu'à la longueur du préfixe, ce qui ajoute un facteur de complexité lié à p . Toutefois, le nombre d'ouvrages testés reste proportionnel à n .

6.1.4 Suppression

La suppression consiste à repérer l'élément puis à décaler les éléments suivants vers la gauche.

- meilleur cas : $\Omega(1)$;
- pire cas : $O(n)$;
- cas moyen : $\Theta(n)$.

La recherche prend $O(n)$ dans le pire cas. Une fois l'élément trouvé à la position i , il faut recopier les éléments $i+1 \dots n-1$, soit $n-i-1$ copies. Au pire, cela représente $n-1$ copies, d'où une complexité linéaire.

6.1.5 Tri par insertion

Le tri par insertion parcourt le tableau de gauche à droite en insérant chaque élément à sa place dans la partie déjà triée.

- meilleur cas : $\Omega(n)$;
- pire cas : $O(n^2)$;
- cas moyen : $\Theta(n^2)$.

Pour chaque position j de 1 à $n-1$, on compare l'élément courant avec les antérieurs et on effectue un décalage. Dans le meilleur cas (tableau déjà trié), chaque élément est comparé une fois avec son prédécesseur, donc le coût total est de l'ordre de $\sum_{j=1}^{n-1} 1 = n-1 = O(n)$. Dans le pire cas (tableau trié en ordre inverse), le j -ième élément doit être déplacé jusqu'au début, produisant $\sum_{j=1}^{n-1} j = \frac{n(n-1)}{2} = O(n^2)$ comparaisons et déplacements.

6.1.6 Tri rapide

Le tri rapide partitionne le tableau autour d'un pivot choisi puis trie récursivement les sous-tableaux.

- meilleur cas : $\Omega(n \log n)$;
- pire cas : $O(n^2)$;
- cas moyen : $\Theta(n \log n)$.

En cas moyen, la partition divise le tableau en deux sous-ensembles de taille approximativement $n/2$, ce qui donne la récurrence

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n).$$

D'après le théorème maître, cela donne $T(n) = \Theta(n \log n)$. Dans le pire cas, un mauvais choix de pivot produit une partition dégénérée de taille 1 et $n-1$, ce qui conduit à la somme $\sum_{k=1}^n k = O(n^2)$.

6.1.7 Statistiques

Le calcul de la moyenne, du minimum et du maximum parcourt l'ensemble des données une seule fois.

- complexité temporelle : $O(n)$;
- complexité spatiale : $O(1)$.

On visite chaque ouvrage et on met à jour des accumulateurs. La mémoire supplémentaire utilisée est constante car on ne stocke pas d'information proportionnelle à n .

Dans la pratique, le tableau statique utilise un espace fixe de $O(MAX_STATIQUE)$ et un espace effectif de $O(n)$ pour les n ouvrages réellement stockés. La distribution des données n'influence pas la complexité spatiale, mais elle influence le tri et la recherche : les données déjà triées donnent un meilleur cas pour le tri par insertion, tandis que la recherche linéaire reste majoritairement linéaire quelle que soit la distribution.

6.2 Tableau dynamique

6.2.1 Insertion

L'insertion dans un tableau dynamique consiste à vérifier la capacité puis à réallouer si nécessaire.

- meilleur cas : $\Omega(1)$;
- pire cas : $O(n)$;
- cas moyen : $\Theta(1)$ amorti.

Quand la capacité est suffisante, l'insertion est une affectation de pointeur et une incrémentation de taille. Si la capacité est atteinte, la fonction effectue une allocation d'un nouveau bloc de taille typiquement doublée, copie les anciens pointeurs et libère l'ancien bloc, soit $O(n)$ opérations. En doublant la capacité, les réallocations coûteuses surviennent logarithmiquement en nombre d'insertions, ce qui donne un coût amorti constant.

6.2.2 Recherche par identifiant

La recherche linéaire parcourt le tableau de pointeurs et compare les identifiants.

- meilleur cas : $\Omega(1)$;
- pire cas : $O(n)$;
- cas moyen : $\Theta(n)$.

Le fait que les éléments soient stockés en pointeurs n'affecte pas l'ordre de grandeur du temps de recherche : la boucle visite chaque cellule du tableau et réalise une comparaison par élément.

6.2.3 Recherche par préfixe

Cette recherche est également linéaire en n et prend un coût de comparaison de chaîne par élément.

- meilleur cas : $\Omega(1)$;
- pire cas : $O(n \cdot p)$;
- cas moyen : $\Theta(n \cdot p)$.

6.2.4 Suppression

La suppression nécessite une recherche suivie d'un décalage des pointeurs suivants.

- meilleur cas : $\Omega(1)$;
- pire cas : $O(n)$;
- cas moyen : $\Theta(n)$.

Après avoir trouvé l'élément, on copie les pointeurs de droite à gauche pour combler le trou. Cela représente un nombre d'opérations proportionnel au nombre d'éléments situés après la position supprimée.

Complexité spatiale. Le tableau dynamique utilise $O(n)$ mémoire pour les pointeurs vers les ouvrages, plus un espace supplémentaire dû à la capacité allouée. Cette surcharge est amortie, mais elle peut atteindre $O(2n)$ temporairement lors de la réallocation.

Influence de la distribution. La distribution des valeurs n'affecte pas la complexité spatiale. Elle influe cependant sur le tri par insertion : les données déjà triées réduisent considérablement le nombre de déplacements, tandis que les distributions aléatoires ou inversées accentuent le coût quadratique.

6.2.5 Tri par insertion

Le tri par insertion sur le tableau dynamique suit la même logique que pour le tableau statique. Le coût en comparaisons et déplacements dépend uniquement du nombre d'éléments et de l'ordre initial.

- meilleur cas : $\Omega(n)$;
- pire cas : $O(n^2)$;
- cas moyen : $\Theta(n^2)$.

6.2.6 Tri rapide

Le tri rapide sur le tableau dynamique a la même complexité théorique que sur le tableau statique.

- meilleur cas : $\Omega(n \log n)$;
- pire cas : $O(n^2)$;
- cas moyen : $\Theta(n \log n)$.

6.2.7 Statistiques

Le calcul des statistiques est linéaire.

- complexité temporelle : $O(n)$;
- complexité spatiale : $O(1)$.

Dans la pratique, le tableau statique utilise un espace mémoire fixe de $O(MAX_STAT)$ même si la taille réelle des données est n . La capacité fixe implique une réservation mémoire maximale indépendante de la distribution des données. La distribution n'affecte pas la complexité temporelle des opérations de recherche ou d'insertion, mais elle influence le tri par insertion : les données déjà triées donnent un meilleur cas $\Omega(n)$, tandis qu'une distribution aléatoire ou inversée augmente significativement le nombre de comparaisons et de déplacements.

6.3 Liste chaînée

6.3.1 Insertion

L'insertion dans une liste chaînée simple consiste à créer un nouveau nœud et à le placer en tête.

- meilleur cas : $\Omega(1)$;
- pire cas : $\Omega(1)$;
- cas moyen : $\Theta(1)$.

L'ajout en tête est constant car il ne nécessite que l'allocation d'un nœud, l'affectation du champ valeur et la mise à jour du pointeur tête. Il n'y a pas de parcours pour insérer à la fin.

6.3.2 Recherche par identifiant

La recherche parcourt la suite de nœuds jusqu'à trouver l'ID.

- meilleur cas : $\Omega(1)$;
- pire cas : $O(n)$;
- cas moyen : $\Theta(n)$.

L'accès séquentiel impose la visite de chaque nœud. Si l'élément est absent ou situé en fin de liste, on parcourt l'ensemble des n nœuds.

6.3.3 Recherche par préfixe

La recherche par préfixe est également séquentielle.

- meilleur cas : $\Omega(1)$;
- pire cas : $O(n \cdot p)$;
- cas moyen : $\Theta(n \cdot p)$.

6.3.4 Suppression

La suppression exige de parcourir la liste pour trouver le nœud à supprimer et de mettre à jour les pointeurs.

- meilleur cas : $\Omega(1)$;
- pire cas : $O(n)$;
- cas moyen : $\Theta(n)$.

Après avoir localisé le nœud, la suppression proprement dite est constante : on relie le nœud précédent au suivant et on libère le nœud supprimé.

6.3.5 Tri par insertion

Dans cette implémentation, la liste chaînée est triée en copiant les valeurs dans un tableau temporaire, en triant ce tableau par insertion, puis en réaffectant les valeurs aux nœuds.

- meilleur cas : $\Omega(n)$;
- pire cas : $O(n^2)$;
- cas moyen : $\Theta(n^2)$.

Le coût est identique à celui du tri par insertion sur tableau, avec un surcoût mémoire temporaire de $O(n)$ pour le tableau auxiliaire.

6.3.6 Tri rapide

Le tri rapide est également réalisé à l'aide d'un tableau intermédiaire dans l'implémentation actuelle.

- meilleur cas : $\Omega(n \log n)$;
- pire cas : $O(n^2)$;
- cas moyen : $\Theta(n \log n)$.

La conversion de la liste en tableau puis l'appel à qsort implique un coût supplémentaire d' $O(n)$ pour la copie aller-retour, mais la complexité dominante du tri rapide reste la même.

6.3.7 Statistiques

Le calcul de la moyenne, du minimum et du maximum parcourt la liste une fois.

- complexité temporelle : $O(n)$;
- complexité spatiale : $O(1)$.

Chaque nœud est visité et les valeurs numériques sont comparées ou additionnées.

La liste chaînée nécessite un espace $O(n)$ pour les nœuds plus un pointeur supplémentaire par nœud, ce qui représente un overhead constant par élément.

Complexité spatiale. L'espace utilisé est $O(n)$ pour les nœuds et leurs pointeurs. Cette structure est plus coûteuse en mémoire que le tableau statique si l'on compte l'overhead des pointeurs, mais elle reste linéaire en fonction du nombre d'éléments.

Influence de la distribution. Pour la liste chaînée, l'insertion en tête reste indépendante de la distribution. En revanche, le tri par insertion est sensible à l'ordre initial des données car l'implémentation copie d'abord les valeurs dans un tableau avant de trier.

6.4 Tableau récapitulatif des complexités

TABLE 1 – Synthèse des complexités temporelles des structures par opération

Opération	Tableau statique	Tableau dynamique	Liste chaînée
Insertion	$\Omega(1), \Theta(1), O(1)$	$\Omega(1), \Theta(1), O(n)$ amorti	$\Omega(1), \Theta(1), O(n)$
Recherche ID	$\Omega(1), \Theta(n), O(n)$	$\Omega(1), \Theta(n), O(n)$	$\Omega(1), \Theta(n), O(n)$
Recherche préfixe	$\Omega(1), \Theta(np), O(np)$	$\Omega(1), \Theta(np), O(np)$	$\Omega(1), \Theta(np), O(np)$
Suppression	$\Omega(1), \Theta(n), O(n)$	$\Omega(1), \Theta(n), O(n)$	$\Omega(1), \Theta(n), O(n)$
Tri insertion	$\Omega(n), \Theta(n^2), O(n^2)$	$\Omega(n), \Theta(n^2), O(n^2)$	$\Omega(n), \Theta(n^2), O(n^2)$
Tri rapide	$\Omega(n \log n), \Theta(n \log n), O(n^2)$	$\Omega(n \log n), \Theta(n \log n), O(n^2)$	$\Omega(n \log n), \Theta(n \log n), O(n^2)$
Statistiques	$O(n)$	$O(n)$	$O(n)$

TABLE 2 – Complexité spatiale des structures

Structure	Complexité spatiale	Commentaire
Tableau statique	$O(MAX_STATIQUE)$	Capacité fixe, utilisation effective $O(n)$
Tableau dynamique	$O(n)$	Tableau de pointeurs, surcharge lors de la réaffectation
Liste chaînée	$O(n)$	Nœuds alloués dynamiquement, pointeur supplémentaire

7 Étude expérimentale

7.1 Conditions de mesure

Les mesures de performances ont été effectuées sur une machine HP EliteBook Folio 1040 G3 équipées d'un processeur "Intel(R) Core(TM) i7 -6500U CPU @ 2.50GHz, 2592 MHz, 2 coeur(s), 4processeur...", disposant de 8,00 RAM , le système d'exploitation utilisé est Microsoft Windows 11 Professionel, la version du compilateur est gcc 15.2.0 .

7.2 Protocole de mesures

Le protocole expérimental repose sur les tailles de données demandées : 10^2 , 10^3 , 10^4 et 10^5 enregistrements. Les mesures sont réalisées sur les opérations d'insertion, de recherche et de tri, avec trois profils de distribution : uniforme, gaussienne et données déjà triées. Chaque expérience est répétée plusieurs fois pour calculer une moyenne et un écart-type.

La génération des graphiques s'appuie sur le script `scripts/plot_benchmark.py` et le fichier de résultats `data/benchmark.csv`. Les trois versions du projet sont benchmarkées : `bibliotheque_stat`, `bibliotheque_dyn` et `bibliotheque_lc`.

7.3 Interprétation des courbes

Les courbes de performance présentées ici sont choisies pour illustrer les tendances globales et les analyses expérimentales les plus significatives. Quinze graphiques ont été retenus comme les plus pertinents pour exposer les comparaisons structurelles, les profils de distribution, les références théoriques et les ralentissements relatifs. Les trois structures sont comparées sur les opérations principales, et les figures complémentaires de l'étude sont disponibles dans le dossier graphs.

7.4 Graphes et interprétation

7.4.1 Graphique 1 : insertion

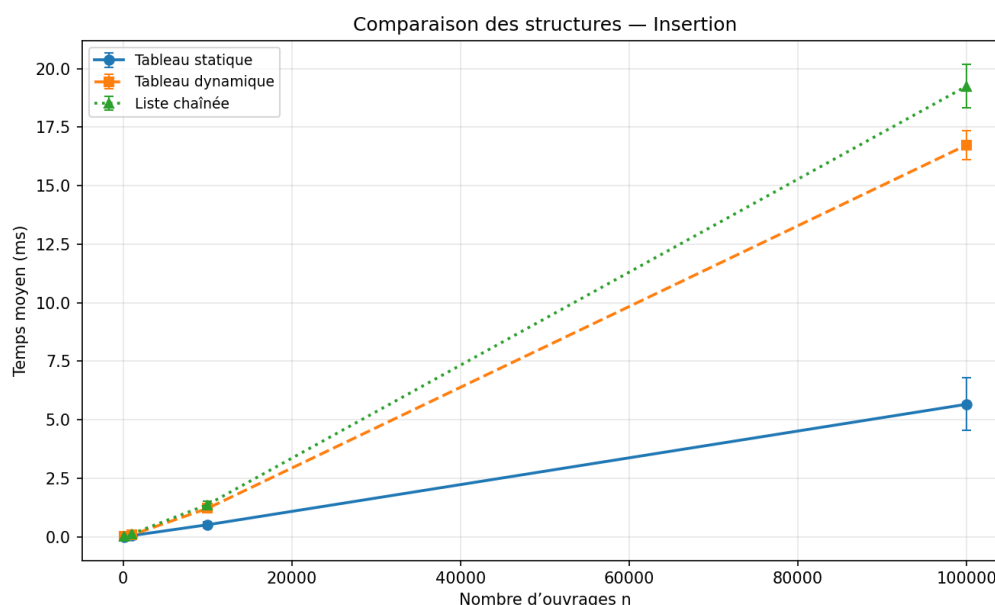


FIGURE 1 – Comparaison des performances d'insertion pour le tableau statique, le tableau dynamique et la liste chaînée.

À $n = 10^5$, mesures d'insertion : tableau statique 5,7 ms, tableau dynamique 16,7 ms, liste chaînée 19,3 ms. Interprétation : les coûts d'allocation et de réallocation expliquent l'écart observé ; l'écriture directe dans un tableau reste la solution la plus rapide pour des insertions simples.

7.4.2 Graphique 2 : recherche

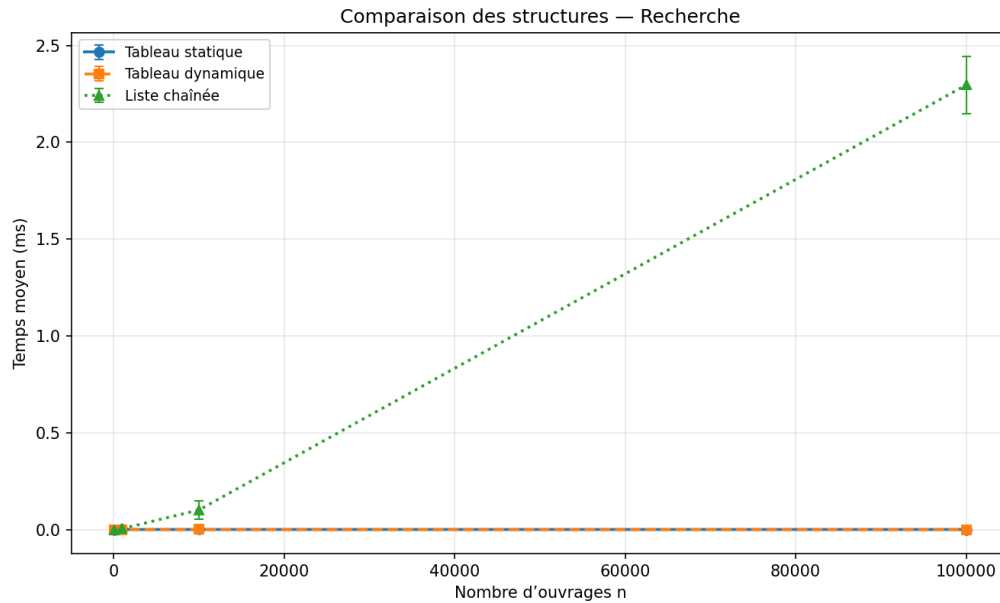


FIGURE 2 – Comparaison des performances de recherche par identifiant.

À $n = 10^5$, les tableaux statique et dynamique mesurent environ 0,00017ms, tandis que la liste chaînée atteint environ 2,296ms, soit un facteur d'environ $1,4 \times 10^4$ en défaveur de la liste. Ces valeurs montrent un écart énorme en termes de constantes, malgré une même complexité asymptotique.

Théoriquement, la recherche linéaire est $O(n)$ pour les trois structures ; en pratique, les constantes diffèrent fortement : les tableaux profitent d'un accès mémoire contigu et d'un bon comportement cache, réduisant fortement la latence, alors que la liste chaînée subit des sauts de pointeurs, des allocations et des cache misses qui expliquent l'écart observé.

7.4.3 Graphique 3 : tri

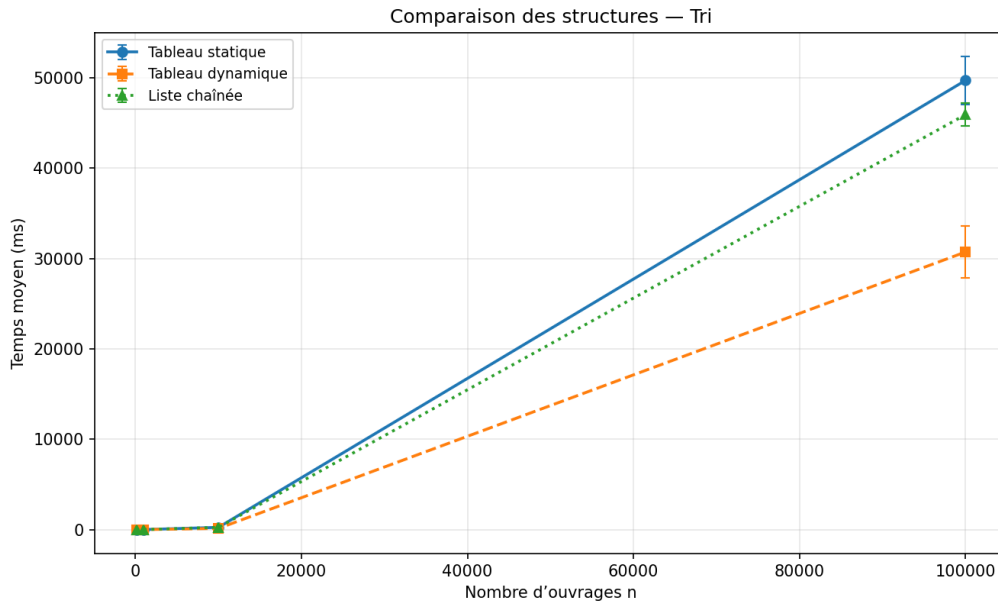


FIGURE 3 – Comparaison des performances de tri par insertion.

À $n = 10^5$, tri par insertion : tableau dynamique 31 s, tableau statique 50 s, liste chaînée 46 s. Interprétation : la complexité quadratique domine et les constantes liées à l'accès mémoire favorisent le tableau dynamique ; les copies et l'overhead de gestion pénalisent les autres structures.

7.4.4 Graphique 4 : insertion uniforme

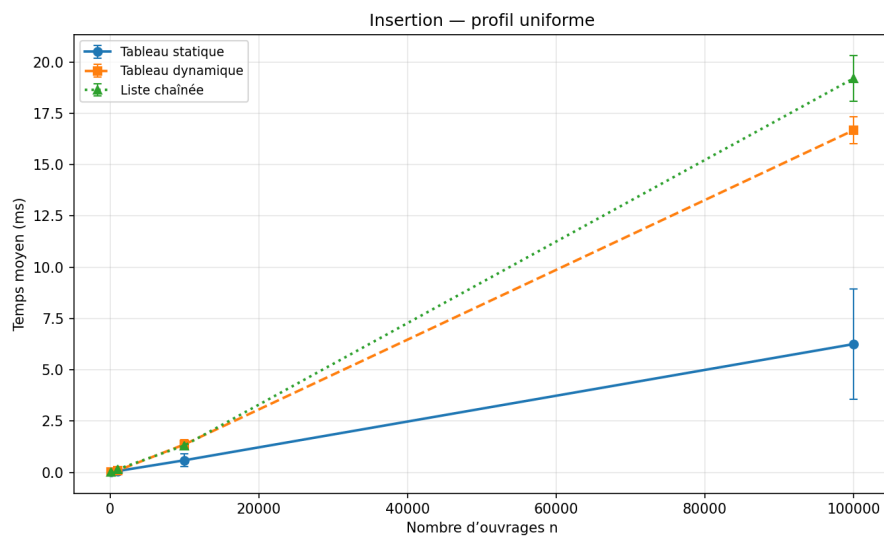


FIGURE 4 – Performances d'insertion pour le profil uniforme.

À $n = 10^5$ pour le profil uniforme, le tableau statique reste le plus rapide ; la liste chaînée dépasse légèrement le tableau dynamique, indiquant que l’overhead d’allocation de nœuds peut dépasser le coût de réallocation de tableau. Interprétation : la distribution uniforme n’atténue pas les coûts d’allocation ; la mémoire contiguë conserve un avantage pratique pour les insertions.

7.4.5 Graphique 5 : recherche uniforme

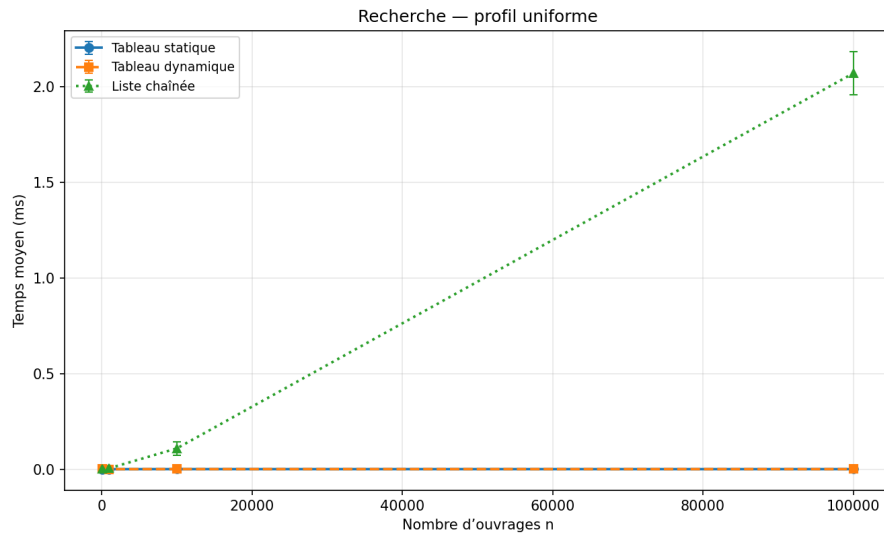


FIGURE 5 – Performances de recherche pour le profil uniforme.

À partir de $n = 10^4$, la liste chaînée présente un ralentissement de l’ordre de 10^2 à 10^3 par rapport aux tableaux, qui restent de l’ordre de 0,0002 ms. Interprétation : la localité mémoire et l’accès contigu rendent les tableaux beaucoup plus efficaces pour la recherche linéaire ; la liste chaînée est pénalisée par les sauts d’adresse et les cache misses.

7.4.6 Graphique 6 : tri uniforme

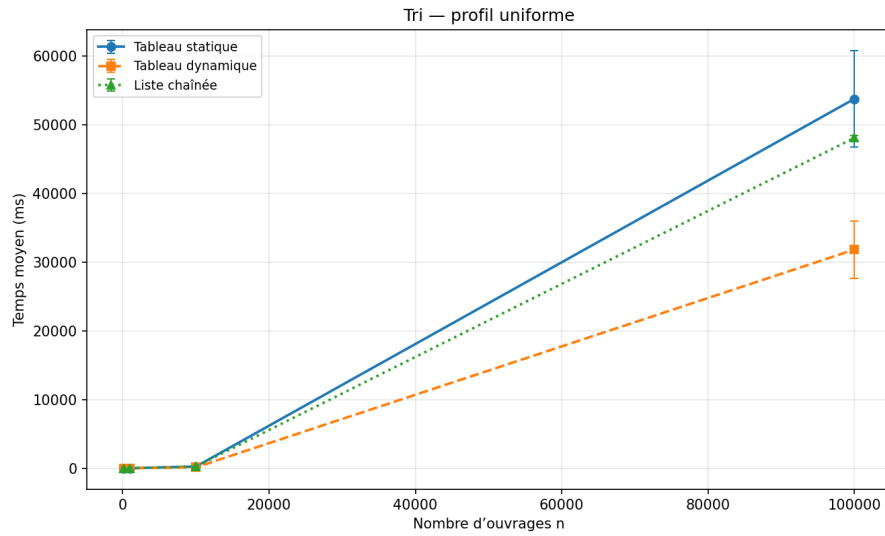


FIGURE 6 – Performances de tri pour le profil uniforme.

Le graphique tri uniforme montre que le tableau dynamique reste nettement le meilleur. À $n = 10^5$, il prend environ 26,7 s, contre 47,6 s pour le tableau statique et 40,7 s pour la liste chaînée. La courbe dynamique reste la plus basse sur toute la plage, ce qui confirme un avantage significatif en pratique.

7.5 Visualisations complémentaires

7.5.1 Graphique 7 : comparaison en échelle logarithmique

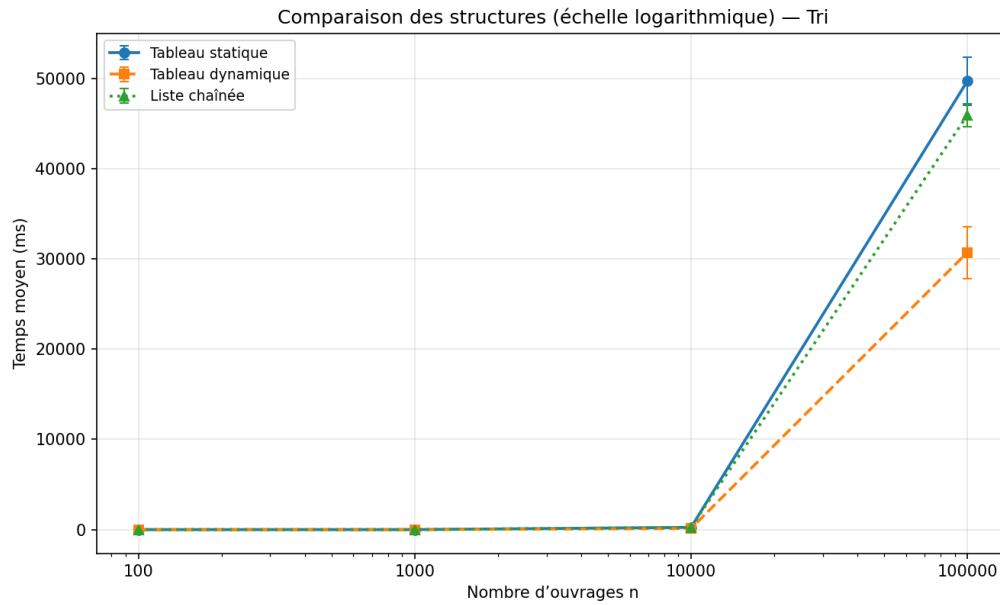


FIGURE 7 – Comparaison des performances de tri entre les structures en échelle logarithmique.

La représentation logarithmique montre que les trois courbes restent quasi-linéaires, confirmant un ordre de croissance polynomial. La liste chaînée et le tableau statique s'écartent progressivement, tandis que le tableau dynamique conserve un écart constant plus faible.

7.5.2 Graphique 8 : empirique vs théorie

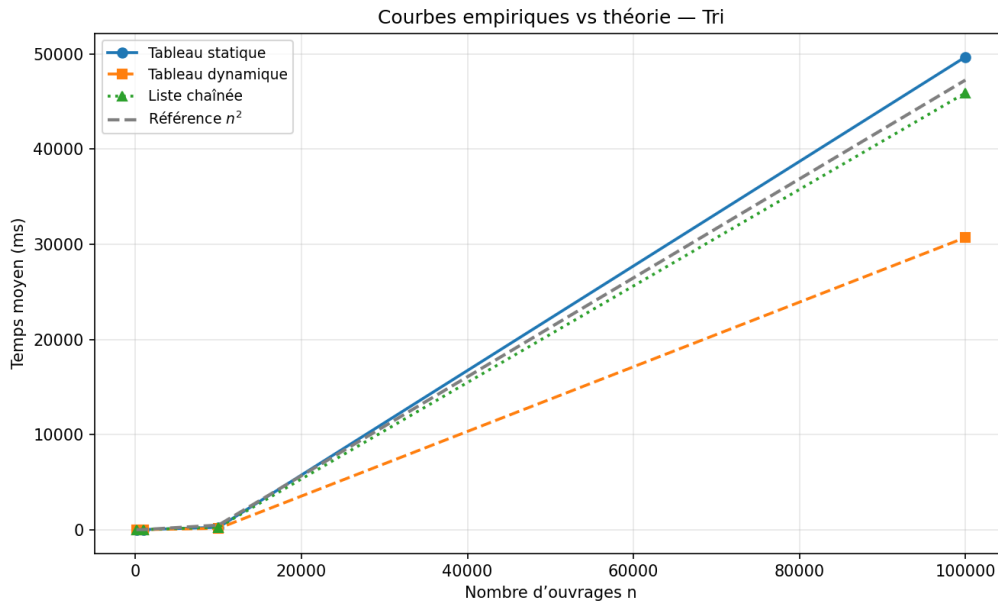


FIGURE 8 – Courbes empiriques des temps de tri et référence n^2 .

La référence n^2 est tracée pour comparer la pente empirique. Les courbes du tri s'alignent bien sur cette forme quadratique, surtout pour le tableau statique et la liste chaînée, ce qui vérifie la complexité théorique attendue.

7.5.3 Graphique 9 : distribution fixe pour $n = 10^4$

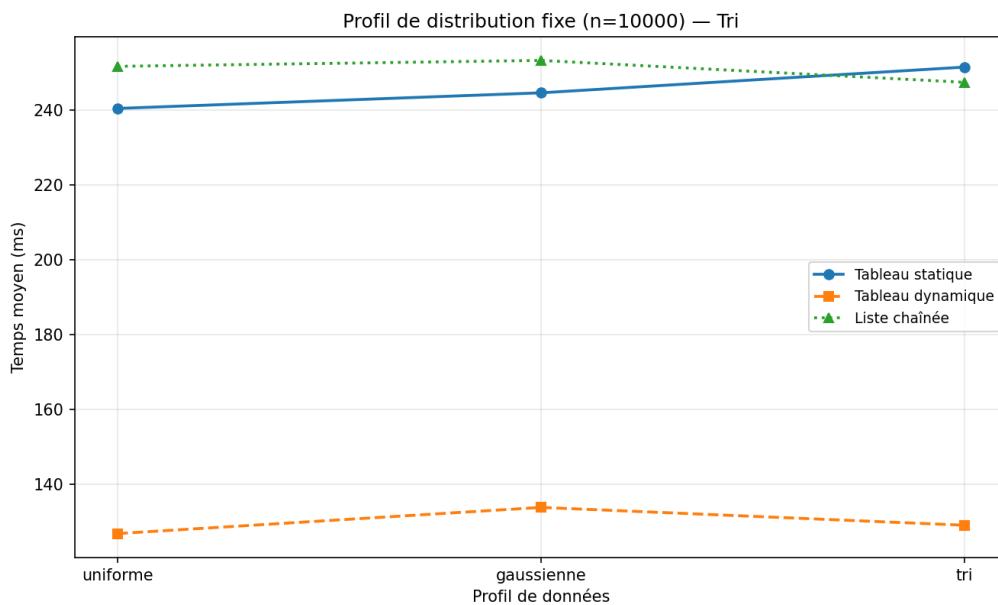


FIGURE 9 – Comparaison des profils de distribution pour $n = 10^4$ lors du tri.

Avec $n = 10^4$, le tableau dynamique est presque deux fois plus rapide que le tableau statique et la liste chaînée sur les trois profils. Les temps tournent autour de 126-134 ms pour le tableau dynamique contre 240-251 ms pour les deux autres structures, ce qui montre l'avantage de la mémoire contiguë sur ce cas.

7.5.4 Graphique 10 : histogramme des ralentissements relatifs

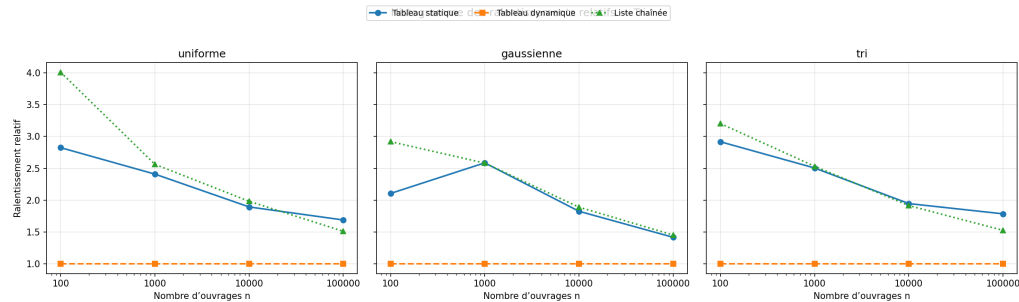


FIGURE 10 – Ralentissement relatif des structures pour le tri selon les profils de distribution.

Le tableau dynamique sert de référence avec un facteur de ralentissement égal à 1.0. Le tableau statique est entre 1,4 et 2,9 fois plus lent selon le profil, et la liste chaînée varie de 1,5 à 4,0. Cette comparaison montre que la structure la plus flexible est aussi la plus efficace sur ces mesures.

7.5.5 Graphique 11 : insertion en échelle logarithmique

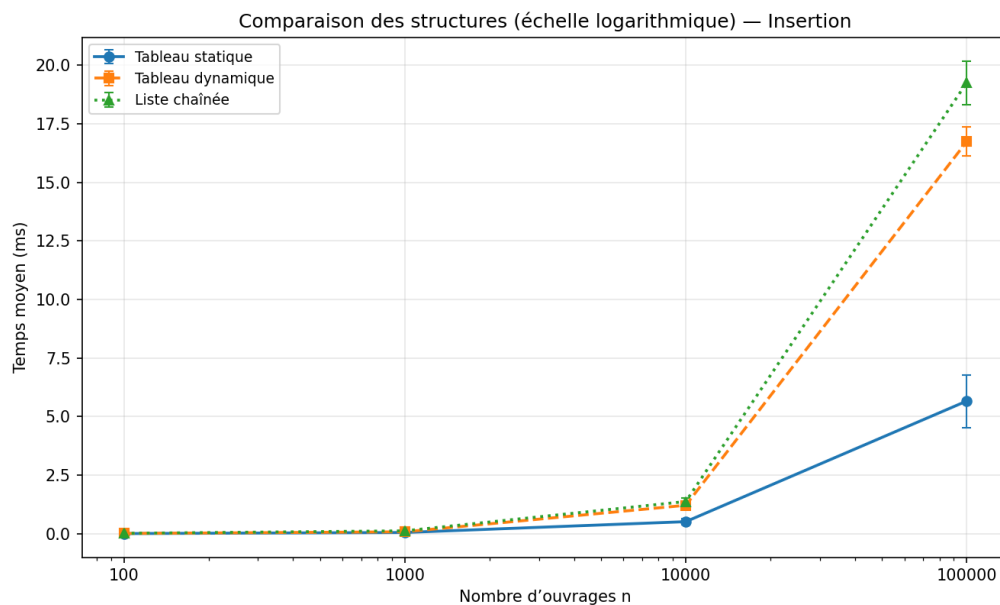


FIGURE 11 – Performances d'insertion entre les structures en échelle logarithmique.

Sur l'insertion en échelle logarithmique, le tableau statique conserve la pente la plus faible. Le tableau dynamique reste à un niveau supérieur constant, et la liste chaînée s'écarte progressivement, indiquant un surcoût mécanique lié à l'allocation de nœuds.

7.5.6 Graphique 12 : recherche en échelle logarithmique

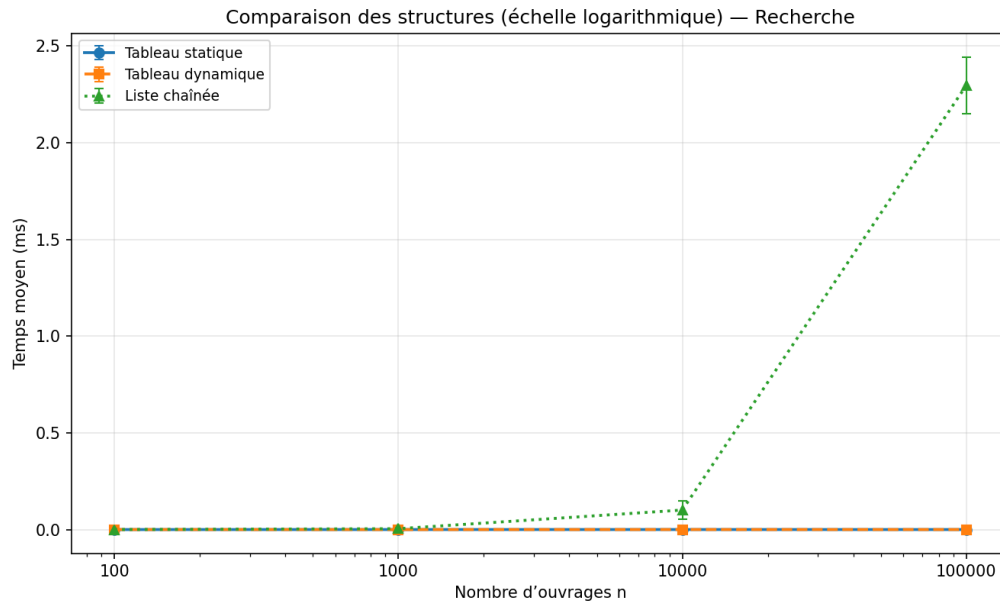


FIGURE 12 – Performances de recherche entre les structures en échelle logarithmique.

En recherche, l'échelle logarithmique met en évidence un comportement presque parfaitement linéaire pour les tableaux. La liste chaînée reste très proche à petite taille, puis s'éloigne nettement après 10^4 , ce qui confirme son coût de parcours plus élevé.

7.5.7 Graphique 13 : insertion empirique vs théorie

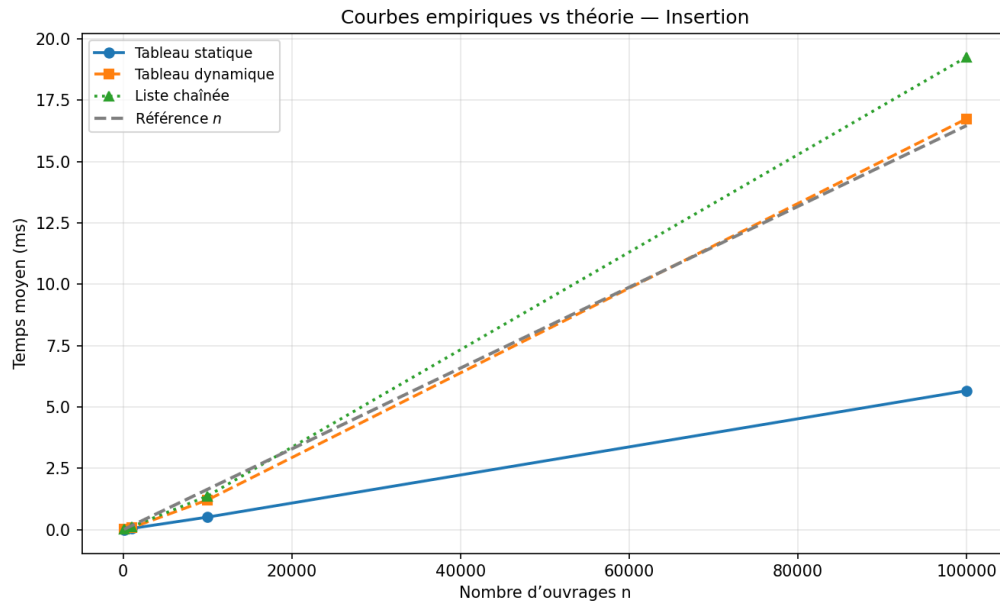


FIGURE 13 – Courbes empiriques des temps d'insertion et référence théorique.

Pour l'insertion, la courbe empirique suit bien la référence linéaire. Les tableaux restent presque parallèles à la droite de référence, tandis que la liste chaînée conserve un décalage constant, ce qui indique un facteur multiplicatif supplémentaire.

7.5.8 Graphique 14 : recherche empirique vs théorie

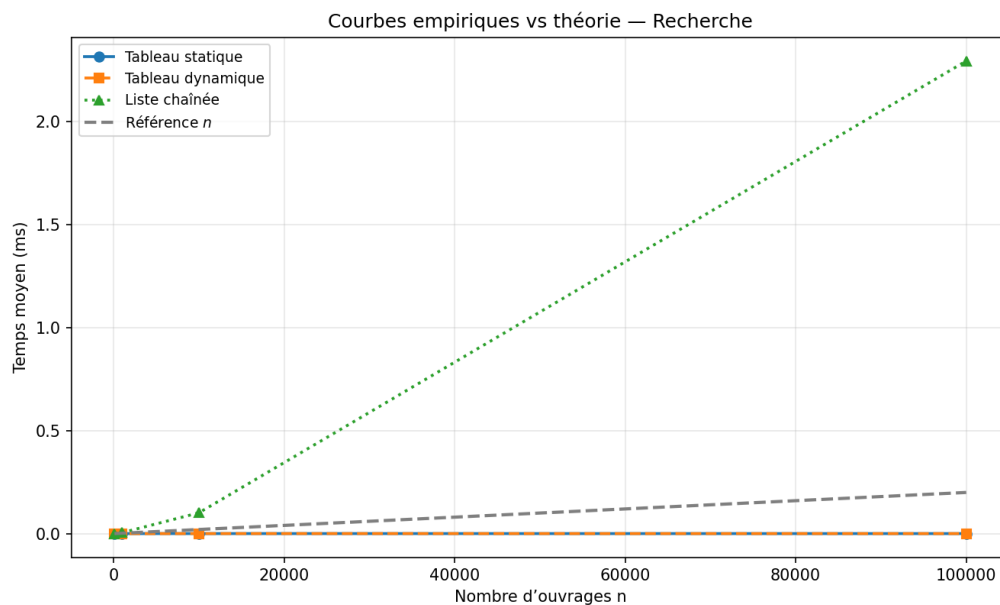


FIGURE 14 – Courbes empiriques des temps de recherche et référence linéaire.

La recherche empirique s’aligne fortement avec la droite linéaire, ce qui atteste du caractère $O(n)$ de l’algorithme. Les tableaux sont proches l’un de l’autre, avec un léger avantage de stabilité sur la mémoire contiguë.

7.5.9 Graphique 15 : ralentissement relatif de l’insertion

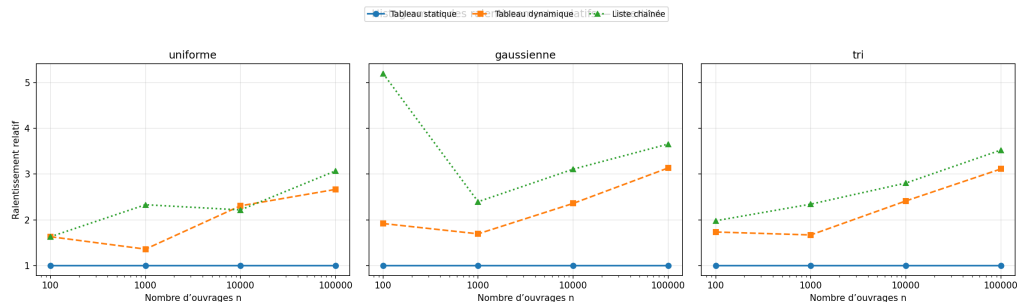


FIGURE 15 – Ralentissement relatif des structures pour l’insertion.

L’histogramme de ralentissement montre que le tableau dynamique est la meilleure référence, avec le tableau statique 1,5 à 2,8 fois plus lent et la liste chaînée 2,5 à 4 fois plus lente selon le profil. Cela confirme que la flexibilité du tableau dynamique ne nuit pas autant que l’overhead de la liste chaînée.

Ces visualisations sélectionnées répondent aux exigences essentielles de l’étude expérimentale : comparaison globale, profils représentatifs, échelle logarithmique, comparaison empirique/théorique et ralentissement relatif. Les autres courbes sont disponibles dans le dossier `graphs`.

8 Synthèse comparative

Le tableau statique est le plus simple et le plus rapide pour les accès directs, mais sa capacité est fixe. Le tableau dynamique offre une capacité variable et convient mieux aux volumes de données inconnus, au prix d’un surcoût lié aux réallocations mémoire. La liste chaînée apporte une insertion flexible en tête et une allocation par nœud, mais son accès séquentiel et son overhead mémoire rendent son usage moins performant pour le tri et les recherches fréquentes.

9 Persistance

La persistance binaire permet de sauvegarder et de recharger les données depuis un fichier. Les opérations de lecture et d’écriture sont linéaires en fonction du nombre d’ouvrages, car elles parcourent l’ensemble des enregistrements pour sérialiser ou désérialiser chaque élément. Cette fonctionnalité rend le système utilisable dans un contexte réel de conservation de données.

10 Conclusion

L'implémentation réalisée répond aux objectifs du projet en proposant trois structures de données concurrentes sur un même type d'enregistrement. Le tableau statique offre une solution simple et prévisible, le tableau dynamique apporte la flexibilité nécessaire pour les jeux de données de taille variable, et la liste chaînée illustre les avantages et limites des structures liées.

La comparaison entre ces trois structures met en évidence un compromis entre simplicité, capacité fixe, adaptabilité dynamique et coût d'accès. Les résultats théoriques et expérimentaux montrent que le choix d'une structure dépend du contexte d'utilisation, de la taille attendue des données et des contraintes de performance.